

Министерство образования Республики Беларусь

Учреждение высшего образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Кафедра электронных вычислительных машин

Отчет по лабораторной работе № 2
Тема: «Проектирование цифровых фильтров»

Выполнил:
студент группы 250541
Власов Р. Е.

Проверил:
Перцев Д. Ю.

Минск 2026

1 Исходные данные

Цель лабораторной работы — на основе сигнала из лабораторной работы № 1 спроектировать и проверить работу однородного, КИХ и БИХ цифровых фильтров. Варианты: ЛР1 — БПФ_Ч 0, то есть БПФ с прореживанием по частоте; ЛР2 — КИХ-фильтр типа Р с прямоугольным окном и БИХ однополосный НЧ-фильтр.

Таблица 1 – Исходные данные

Параметр	Значение
Базовый сигнал	Первый периодический сигнал из ЛР1
Частота дискретизации	2048 Гц
Длительность сигнала	2,0 с
Количество отсчетов	4096
Искажение	Синусоидальная помеха 700 Гц с амплитудой 0,35 и белый шум 0,03
Однородный фильтр	Скользящее среднее, окно $M = 9$
КИХ-фильтр	Режекторный фильтр, прямоугольное окно, полоса подавления 650...750 Гц, 101 коэффициент
БИХ-фильтр	Однополосный НЧ-фильтр, частота среза 450 Гц

2 Описание алгоритма работы программы

2.1 Однородный фильтр

Однородный фильтр реализован как КИХ-фильтр скользящего среднего. Все коэффициенты одинаковы, поэтому каждый выходной отсчет является средним значением соседних входных отсчетов.

$$h[n] = 1/M, \quad n = 0, 1, \dots, M-1$$

$$y[n] = (1/M) \cdot \sum x[n-k], \quad M = 9$$

Для выбранного окна $M = 9$ коэффициенты однородного фильтра равны:

$$h[n] = [0,111111; 0,111111; 0,111111; 0,111111; 0,111111; 0,111111; 0,111111; 0,111111; 0,111111]$$

Такой фильтр уменьшает случайные высокочастотные колебания, но одновременно сглаживает полезный сигнал, поэтому его результат необходимо оценивать по временной форме сигнала.

2.2 КИХ-фильтр Р с прямоугольным окном

КИХ-фильтр спроектирован как полосо-заграждающий фильтр для подавления внесенной помехи около 700 Гц. Используется прямоугольное окно, поэтому рассчитанная идеальная импульсная характеристика не дополнительно взвешивается: $w[n] = 1$.

$$\begin{aligned}\omega_1 &= 2\pi f_1/F_s, \quad \omega_2 = 2\pi f_2/F_s \\ h[M] &= 1 - (\omega_2 - \omega_1)/\pi \\ h[n] &= -(\sin(\omega_2 k) - \sin(\omega_1 k))/(\pi k), \quad k = n - M, \quad k \neq 0\end{aligned}$$

Для заданной полосы подавления $f_1 = 650$ Гц и $f_2 = 750$ Гц при $F_s = 2048$ Гц нормированные круговые частоты равны:

$$\omega_1 = 2\pi \cdot 650/2048 = 1,9942 \text{ рад/отсчет}; \quad \omega_2 = 2\pi \cdot 750/2048 = 2,3010 \text{ рад/отсчет}$$

Так как число коэффициентов равно 101, центральный индекс $M = (101 - 1)/2 = 50$. Центральный коэффициент рассчитан как:

$$h[50] = 1 - (\omega_2 - \omega_1)/\pi = 0,90234375$$

Остальные коэффициенты симметричны относительно центрального отсчета. Прямоугольное окно не изменяет рассчитанную идеальную импульсную характеристику, поэтому форма АЧХ определяется выбранной длиной фильтра и полосой подавления.

2.3 БИХ однополюсный НЧ-фильтр

БИХ-фильтр имеет рекурсивную структуру: текущий выход зависит от текущего входа и предыдущего выходного отсчета. Такой фильтр сглаживает сигнал и ослабляет высокочастотные составляющие, включая добавленную помеху и шум.

$$\begin{aligned}\alpha &= \exp(-2\pi f_c/F_s) = 0,25143200 \\ y[n] &= (1 - \alpha) \cdot x[n] + \alpha \cdot y[n-1]\end{aligned}$$

Для $f_c = 450$ Гц и $F_s = 2048$ Гц коэффициент α равен:

$$\alpha = \exp(-2\pi \cdot 450/2048) = 0,25143200$$

Следовательно, вклад текущего входного отсчета составляет $1 - \alpha = 0,74856800$, а вклад предыдущего выходного отсчета — $0,25143200$. Благодаря рекурсивной связи фильтр имеет бесконечную импульсную характеристику.

3 Порядок выполнения лабораторной работы

На первом этапе был взят базовый периодический сигнал из лабораторной работы № 1 с частотой дискретизации 2048 Гц и длительностью 2 секунды.

На втором этапе в сигнал была внесена контролируемая помеха: синусоида частотой 700 Гц и слабый белый шум. Такое искажение позволяет проверить, подавляют ли фильтры нежелательные высокочастотные составляющие.

На третьем этапе рассчитано ядро однородного фильтра скользящего среднего. Для окна $M = 9$ все коэффициенты равны $1/9 = 0,111111$.

На четвертом этапе рассчитаны коэффициенты КИХ-фильтра типа Р с прямоугольным окном. Полоса подавления выбрана 650...750 Гц, чтобы целевая помеха 700 Гц попадала в заграждаемую область.

На пятом этапе вычислен коэффициент α для однополюсного НЧ-фильтра с частотой среза 450 Гц и составлено разностное уравнение фильтра.

На шестом этапе искаженный сигнал был пропущен через каждый из трех фильтров. Для КИХ-фильтров применялась свертка, для БИХ-фильтра — рекурсивный расчет выходных отсчетов.

На седьмом этапе исходный, искаженный и отфильтрованные сигналы сохранены в WAV-файлы, чтобы результаты можно было проверить не только по графикам, но и прослушиванием.

На восьмом этапе построены амплитудно-частотные характеристики фильтров и временные графики входного сигнала, искаженного сигнала и выходов после фильтрации.

На завершающем этапе сформирована сводка с параметрами фильтров, рассчитанными коэффициентами и разностным уравнением БИХ-фильтра.

4 Результаты работы программы

Программа формирует WAV-файлы чистого, искаженного и отфильтрованных сигналов, строит графики АЧХ трех фильтров и временные диаграммы, а также сохраняет численную сводку рассчитанных коэффициентов.

По временным графикам видно, что внесенная помеха делает сигнал более резким и зашумленным относительно чистого входного сигнала. После однородного фильтра сигнал становится сглаженным, но часть быстрых изменений полезного сигнала также подавляется.

КИХ-фильтр с прямоугольным окном целенаправленно подавляет область около 700 Гц, поэтому он лучше соответствует характеру добавленной синусоидальной помехи. Однополюсный НЧ-фильтр уменьшает высокочастотные составляющие в целом, а не только узкую полосу.

На графике АЧХ КИХ-фильтра наблюдается провал в полосе 650...750 Гц. АЧХ однополюсного НЧ-фильтра плавно убывает после частоты среза 450 Гц. Это соответствует назначению фильтров и подтверждает правильность выбранных коэффициентов.

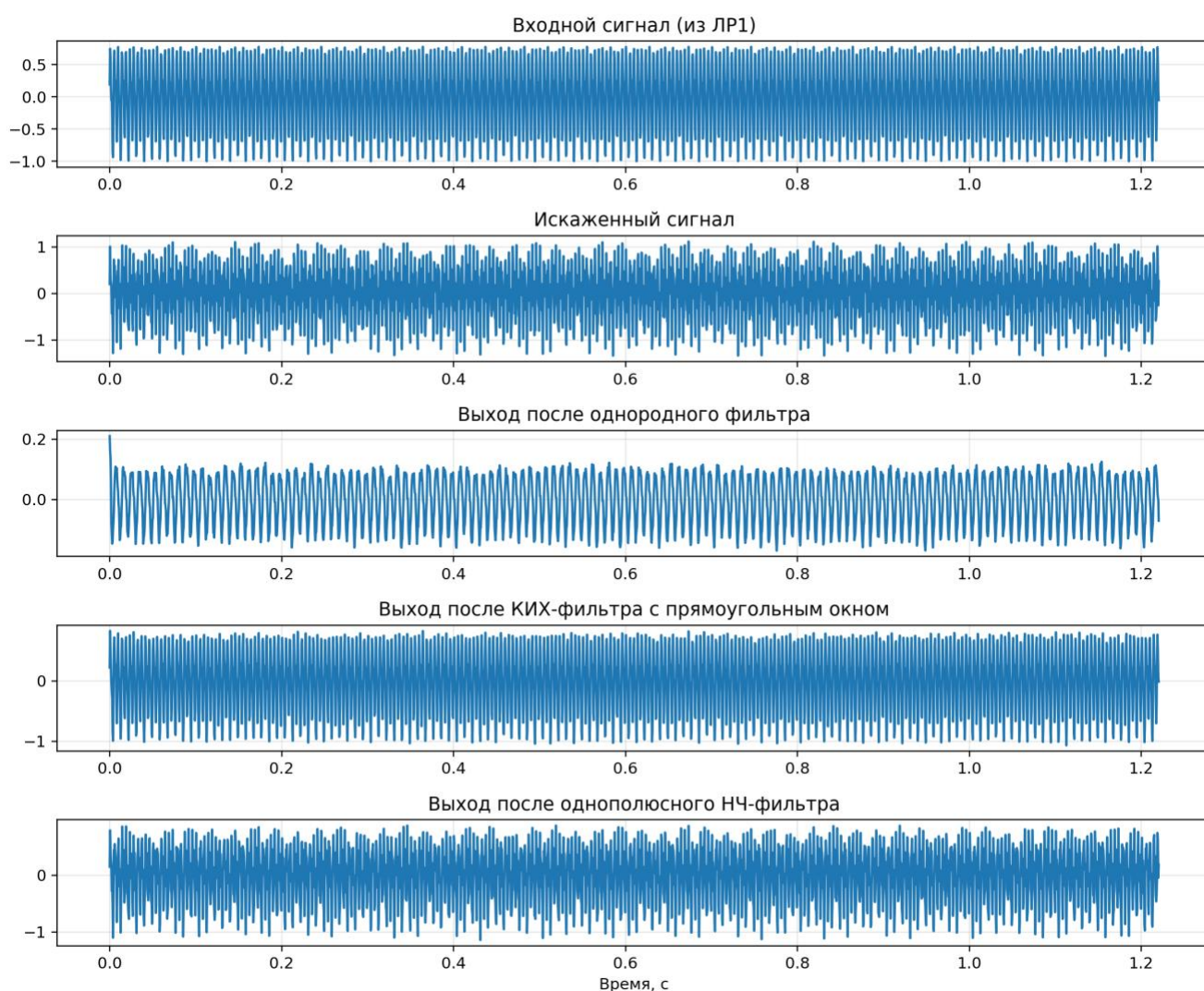


Рисунок 1 – Входной сигнал, искаженный сигнал и результаты фильтрации во временной области

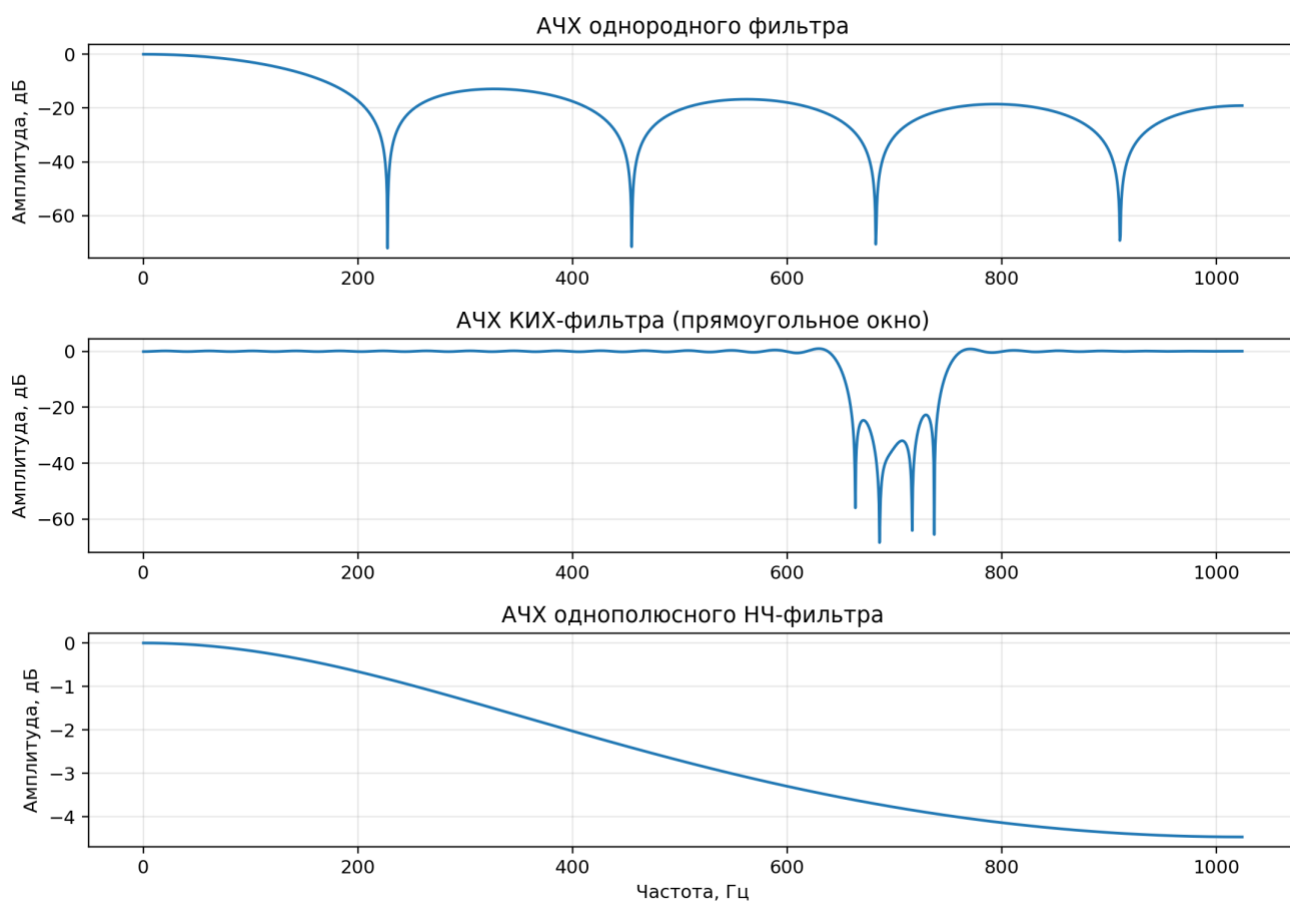


Рисунок 2 – Амплитудно-частотные характеристики однородного, КИХ и БИХ фильтров

Таблица 2 – Расчетные параметры и коэффициенты фильтров

Фильтр	Параметры	Рассчитанные коэффициенты / уравнение
Однородный	$M = 9$	$h[n] = [0,111111; 0,111111; \dots; 0,111111]$
КИХ Р, прямоугольное окно	$f_1 = 650 \text{ Гц}$, $f_2 = 750 \text{ Гц}$, $\text{taps} = 101$	$h[M] = 0,90234375$; первые 10 коэффициентов: $-0,010575$; $0,000150$; $0,009726$; $-0,009954$; $0,001652$; $0,006020$; $-0,006310$; $0,001483$; $0,001481$; $-0,000095$
БИХ однополюсный НЧ	$f_c = 450 \text{ Гц}$	$\alpha = 0,25143200$; $y[n] = (1-\alpha)x[n] + \alpha y[n-1]$

Таблица 3 – Сформированные файлы результата

Файл	Назначение
чистый_сигнал.wav	Входной сигнал из лабораторной работы № 1
искаженный_сигнал.wav	Сигнал после добавления помехи 700 Гц и шума
выход_однородный_фильтр.wav	Результат обработки однородным фильтром
выход_ких_прямоугольное_окно.wav	Результат обработки КИХ-фильтром с прямоугольным окном
выход_бих_однополюсный_нч.wav	Результат обработки однополюсным НЧ-фильтром
ачх_фильтров.png	Графики АЧХ фильтров
сравнение_сигналов_во_времени.png	Временные графики входа, искажения и выходов фильтров

5 Вывод

В ходе лабораторной работы были спроектированы и реализованы три цифровых фильтра: однородный фильтр скользящего среднего, КИХ-фильтр типа Р с прямоугольным окном и БИХ однополюсный НЧ-фильтр. В сигнал из ЛР1 была внесена помеха 700 Гц и белый шум. По АЧХ видно, что КИХ-фильтр формирует заграждение в заданной полосе 650...750 Гц, однородный фильтр сглаживает сигнал, а однополюсный НЧ-фильтр ослабляет высокочастотные компоненты. Полученные графики подтверждают корректность работы спроектированных фильтров. Подробный расчет коэффициентов показал соответствие параметров фильтров заданному варианту: для КИХ-фильтра используется прямоугольное окно и полоса подавления 650...750 Гц, а для БИХ-фильтра — однополюсная рекурсивная схема с $\alpha = 0,25143200$.

Листинг программы

Вспомогательный модуль из ЛР1 signal_io.py

```
001 from __future__ import annotations
002
003 from pathlib import Path
004 import wave
005
006 import numpy as np
007
008
009 INT16_SCALE = 32767
010
011
012 def normalize(signal: np.ndarray) -> np.ndarray:
013     """Нормирует сигнал так, чтобы максимальная амплитуда не превышала
014     1."""
015     peak = np.max(np.abs(signal))
016     if peak == 0:
017         return signal.astype(np.float64)
018     return (signal / peak).astype(np.float64)
019
020 def save_wav_mono_16(path: Path, sample_rate: int, signal: np.ndarray) ->
None:
021     """Сохраняет одномерный сигнал в монофонический 16-битный WAV-файл."""
022     path.parent.mkdir(parents=True, exist_ok=True)
023     normalized = normalize(signal)
024     int_data = np.clip(normalized * INT16_SCALE, -INT16_SCALE,
INT16_SCALE).astype(np.int16)
025
026     with wave.open(str(path), "wb") as wav_file:
027         wav_file.setnchannels(1)
028         wav_file.setsampwidth(2)
029         wav_file.setframerate(sample_rate)
030         wav_file.writeframes(int_data.tobytes())
031
032
033 def read_wav_mono_16(path: Path) -> tuple[int, np.ndarray]:
034     """Считывает 16-битный WAV-файл и возвращает частоту дискретизации и
массив отсчетов."""
035     with wave.open(str(path), "rb") as wav_file:
036         channels = wav_file.getnchannels()
037         sample_rate = wav_file.getframerate()
038         sample_width = wav_file.getsampwidth()
039         frame_count = wav_file.getnframes()
040         raw = wav_file.readframes(frame_count)
041
042         if sample_width != 2:
043             raise ValueError(f"Поддерживается только 16-битный PCM, получен
{sample_width * 8}-битный формат.")
044
045         data = np.frombuffer(raw, dtype=np.int16).astype(np.float64) /
INT16_SCALE
046
047         if channels > 1:
048             data = data[:, :channels]
049
050     return sample_rate, data
051
052
```



```

053 def generate_periodic_signals(sample_rate: int, duration_sec: float) ->
tuple[np.ndarray, np.ndarray]:
054     """Генерирует два периодических тестовых сигнала для лабораторной
работы."""
055     t = np.arange(0.0, duration_sec, 1.0 / sample_rate)
056
057     # Каждый сигнал составлен из нескольких гармоник с разными фазами и
амплитудами.
058     signal_x = (
059         0.70 * np.sin(2 * np.pi * 220 * t)
060         + 0.40 * np.sin(2 * np.pi * 440 * t + 0.3)
061         + 0.20 * np.cos(2 * np.pi * 110 * t + 1.0)
062     )
063     signal_y = (
064         0.80 * np.sin(2 * np.pi * 262 * t + 0.1)
065         + 0.35 * np.cos(2 * np.pi * 392 * t + 0.8)
066         + 0.15 * np.sin(2 * np.pi * 524 * t)
067     )
068
069     return normalize(signal_x), normalize(signal_y)

```

Файл filters.py

```

001 from __future__ import annotations
002
003 import numpy as np
004
005
006 def moving_average_kernel(window_size: int) -> np.ndarray:
007     """Формирует ядро однородного фильтра скользящего среднего."""
008     if window_size <= 0:
009         raise ValueError("Размер окна должен быть положительным.")
010     return np.ones(window_size, dtype=np.float64) / window_size
011
012
013 def design_rectangular_notch_fir(
014     sample_rate: int,
015     stop_low_hz: float,
016     stop_high_hz: float,
017     taps: int,
018 ) -> np.ndarray:
019     """Проектирует режекторный КИХ-фильтр методом прямоугольного окна."""
020     if taps % 2 == 0:
021         raise ValueError("Число коэффициентов taps должно быть нечетным для
симметричного КИХ.")
022     if not (0 < stop_low_hz < stop_high_hz < sample_rate / 2):
023         raise ValueError("Полоса подавления должна лежать в диапазоне (0,
Fs/2).")
024
025     omega1 = 2.0 * np.pi * stop_low_hz / sample_rate
026     omega2 = 2.0 * np.pi * stop_high_hz / sample_rate
027     m = taps // 2
028
029     h = np.zeros(taps, dtype=np.float64)
030     for n in range(taps):
031         k = n - m
032         if k == 0:
033             # Центральный коэффициент получается из предела идеальной
характеристики.
034             h[n] = 1.0 - (omega2 - omega1) / np.pi
035         else:
036             # Идеальная характеристика полосо-заграждающего фильтра.
037             h[n] = -(

```

```

038         (np.sin(omega2 * k) - np.sin(omega1 * k))
039         / (np.pi * k)
040     )
041
042     # Прямоугольное окно: все коэффициенты окна равны единице.
043     window = np.ones(taps, dtype=np.float64)
044     return h * window
045
046
047 def apply_fir(signal: np.ndarray, kernel: np.ndarray) -> np.ndarray:
048     """Применяет КИХ-фильтр к сигналу через свертку."""
049     return np.convolve(signal, kernel, mode="same")
050
051
052 def one_pole_lpf_alpha(sample_rate: int, cutoff_hz: float) -> float:
053     """Вычисляет коэффициент alpha для однополюсного НЧ-фильтра."""
054     if cutoff_hz <= 0:
055         raise ValueError("Частота среза должна быть положительной.")
056     return float(np.exp(-2.0 * np.pi * cutoff_hz / sample_rate))
057
058
059 def apply_one_pole_lpf(signal: np.ndarray, alpha: float) -> np.ndarray:
060     """Применяет рекурсивный однополюсный НЧ-фильтр."""
061     y = np.zeros_like(signal, dtype=np.float64)
062     if len(signal) == 0:
063         return y
064
065     y[0] = (1.0 - alpha) * signal[0]
066     # Текущий выход зависит от текущего входа и предыдущего выходного
    отсчета.
067     for n in range(1, len(signal)):
068         y[n] = (1.0 - alpha) * signal[n] + alpha * y[n - 1]
069
070     return y
071
072
073 def fir_frequency_response(kernel: np.ndarray, sample_rate: int, n_fft: int
= 4096) -> tuple[np.ndarray, np.ndarray]:
074     """Вычисляет АЧХ КИХ-фильтра по БПФ его импульсной характеристики."""
075     h = np.fft.rfft(kernel, n=n_fft)
076     freqs = np.fft.rfftfreq(n_fft, d=1.0 / sample_rate)
077     return freqs, np.abs(h)
078
079
080 def one_pole_lpf_frequency_response(alpha: float, sample_rate: int, n_fft:
int = 4096) -> tuple[np.ndarray, np.ndarray]:
081     """Вычисляет АЧХ однополюсного НЧ-фильтра по аналитической передаточной
    функции."""
082     w = np.linspace(0.0, np.pi, n_fft // 2 + 1)
083     h = (1.0 - alpha) / (1.0 - alpha * np.exp(-1j * w))
084     freqs = (w / (2.0 * np.pi)) * sample_rate
085     return freqs, np.abs(h)

```

Файл plotting.py

```

001 from __future__ import annotations
002
003 from pathlib import Path
004
005 import matplotlib
006
007 matplotlib.use("Agg")
008

```

```

009 import matplotlib.pyplot as plt
010 import numpy as np
011
012
013 def _ensure_dir(path: Path) -> None:
014     """Создает каталог для выходного графика при необходимости."""
015     path.parent.mkdir(parents=True, exist_ok=True)
016
017
018 def plot_time_comparison(
019     clean: np.ndarray,
020     distorted: np.ndarray,
021     homogeneous: np.ndarray,
022     fir: np.ndarray,
023     iir: np.ndarray,
024     sample_rate: int,
025     path: Path,
026     max_samples: int = 2500,
027 ) -> None:
028     """Строит временные графики входа, искаженного сигнала и выходов
029     фильтров."""
030     _ensure_dir(path)
031
032     # Для наглядности выводится начальный фрагмент, а не все 2 секунды
033     # сигнала.
034     samples = min(max_samples, len(clean))
035     t = np.arange(samples) / sample_rate
036
037     plt.figure(figsize=(11, 9))
038
039     plt.subplot(5, 1, 1)
040     plt.plot(t, clean[:samples])
041     plt.title("Входной сигнал (из ЛР1)")
042     plt.grid(True, alpha=0.3)
043
044     plt.subplot(5, 1, 2)
045     plt.plot(t, distorted[:samples])
046     plt.title("Искаженный сигнал")
047     plt.grid(True, alpha=0.3)
048
049     plt.subplot(5, 1, 3)
050     plt.plot(t, homogeneous[:samples])
051     plt.title("Выход после однородного фильтра")
052     plt.grid(True, alpha=0.3)
053
054     plt.subplot(5, 1, 4)
055     plt.plot(t, fir[:samples])
056     plt.title("Выход после КИХ-фильтра с прямоугольным окном")
057     plt.grid(True, alpha=0.3)
058
059     plt.subplot(5, 1, 5)
060     plt.plot(t, iir[:samples])
061     plt.title("Выход после однополюсного НЧ-фильтра")
062     plt.xlabel("Время, с")
063     plt.grid(True, alpha=0.3)
064
065     plt.tight_layout()
066     plt.savefig(path, dpi=220)
067     plt.close()
068
069 def plot_filter_responses(
070     homogeneous_fr: tuple[np.ndarray, np.ndarray],
071     fir_fr: tuple[np.ndarray, np.ndarray],

```

```

071     iir_fr: tuple[np.ndarray, np.ndarray],
072     path: Path,
073 ) -> None:
074     """Строит амплитудно-частотные характеристики трех фильтров."""
075     _ensure_dir(path)
076     h_f, h_a = homogeneous_fr
077     f_f, f_a = fir_fr
078     i_f, i_a = iir_fr
079
080     plt.figure(figsize=(10, 7))
081
082     plt.subplot(3, 1, 1)
083     plt.plot(h_f, 20 * np.log10(np.maximum(h_a, 1e-12)))
084     plt.title("АЧХ однородного фильтра")
085     plt.ylabel("Амплитуда, дБ")
086     plt.grid(True, alpha=0.3)
087
088     plt.subplot(3, 1, 2)
089     plt.plot(f_f, 20 * np.log10(np.maximum(f_a, 1e-12)))
090     plt.title("АЧХ КИХ-фильтра (прямоугольное окно)")
091     plt.ylabel("Амплитуда, дБ")
092     plt.grid(True, alpha=0.3)
093
094     plt.subplot(3, 1, 3)
095     plt.plot(i_f, 20 * np.log10(np.maximum(i_a, 1e-12)))
096     plt.title("АЧХ однополосного НЧ-фильтра")
097     plt.xlabel("Частота, Гц")
098     plt.ylabel("Амплитуда, дБ")
099     plt.grid(True, alpha=0.3)
100
101     plt.tight_layout()
102     plt.savefig(path, dpi=220)
103     plt.close()

```

Файл main.py

```

001 from __future__ import annotations
002
003 from pathlib import Path
004
005 import numpy as np
006
007 from laba1.signal_io import generate_periodic_signals, save_wav_mono_16
008 from laba2.filters import (
009     apply_fir,
010     apply_one_pole_lpf,
011     design_rectangular_notch_fir,
012     fir_frequency_response,
013     moving_average_kernel,
014     one_pole_lpf_alpha,
015     one_pole_lpf_frequency_response,
016 )
017 from laba2.plotting import plot_filter_responses, plot_time_comparison
018
019
020 ROOT = Path(__file__).resolve().parent
021 OUTPUT = ROOT / "output"
022
023 SAMPLE_RATE = 2_048
024 DURATION_SEC = 2.0
025
026 # Параметры варианта:
027 # КИХ: "Р с прямоугольным окном" -> режекторный (полосо-заграждающий) КИХ.

```

```

028 # БИХ: однополюсный НЧ-фильтр.
029 HOMOGENEOUS_WINDOW = 9
030 NOTCH_TAPS = 101
031 NOTCH_LOW_HZ = 650.0
032 NOTCH_HIGH_HZ = 750.0
033 ONE_POLE_CUTOFF_HZ = 450.0
034
035
036 def run() -> None:
037     """Запускает полный эксперимент лабораторной работы 2."""
038     OUTPUT.mkdir(parents=True, exist_ok=True)
039
040     # 1. В качестве входа используется первый периодический сигнал из JPl.
041     clean, _ = generate_periodic_signals(SAMPLE_RATE, DURATION_SEC)
042
043     # 2. В сигнал вносится контролируемая помеха: тон 700 Гц и слабый белый
шум.
044     rng = np.random.default_rng(42)
045     t = np.arange(len(clean)) / SAMPLE_RATE
046     interference = 0.35 * np.sin(2 * np.pi * 700.0 * t)
047     random_noise = 0.03 * rng.normal(size=len(clean))
048     distorted = clean + interference + random_noise
049
050     # 3. Расчет коэффициентов трех фильтров согласно варианту.
051     h_homogeneous = moving_average_kernel(HOMOGENEOUS_WINDOW)
052     h_notch = design_rectangular_notch_fir(
053         sample_rate=SAMPLE_RATE,
054         stop_low_hz=NOTCH_LOW_HZ,
055         stop_high_hz=NOTCH_HIGH_HZ,
056         taps=NOTCH_TAPS,
057     )
058     alpha = one_pole_lpf_alpha(SAMPLE_RATE, ONE_POLE_CUTOFF_HZ)
059
060     # 4. Пропуск искаженного сигнала через однородный, КИХ и БИХ фильтры.
061     y_homogeneous = apply_fir(distorted, h_homogeneous)
062     y_fir = apply_fir(distorted, h_notch)
063     y_iir = apply_one_pole_lpf(distorted, alpha)
064
065     # 5. Сохранение исходного, искаженного и отфильтрованных сигналов.
066     save_wav_mono_16(OUTPUT / "чистый_сигнал.wav", SAMPLE_RATE, clean)
067     save_wav_mono_16(OUTPUT / "искаженный_сигнал.wav", SAMPLE_RATE,
distorted)
068     save_wav_mono_16(OUTPUT / "выход_однородный_фильтр.wav", SAMPLE_RATE,
y_homogeneous)
069     save_wav_mono_16(OUTPUT / "выход_ких_прямоугольное_окно.wav",
SAMPLE_RATE, y_fir)
070     save_wav_mono_16(OUTPUT / "выход_бих_однополюсный_нч.wav", SAMPLE_RATE,
y_iir)
071
072     # 6. Расчет АЧХ нужен для визуальной проверки свойств фильтров.
073     homogeneous_fr = fir_frequency_response(h_homogeneous, SAMPLE_RATE)
074     fir_fr = fir_frequency_response(h_notch, SAMPLE_RATE)
075     iir_fr = one_pole_lpf_frequency_response(alpha, SAMPLE_RATE)
076
077     # 7. Построение графиков АЧХ и временных диаграмм.
078     plot_filter_responses(
079         homogeneous_fr=homogeneous_fr,
080         fir_fr=fir_fr,
081         iir_fr=iir_fr,
082         path=OUTPUT / "ачх_фильтров.png",
083     )
084     plot_time_comparison(
085         clean=clean,
086         distorted=distorted,

```

```

087     homogeneous=y_homogeneous,
088     fir=y_fir,
089     iir=y_iir,
090     sample_rate=SAMPLE_RATE,
091     path=OUTPUT / "сравнение_сигналов_во_времени.png",
092 )
093
094 # 8. Сводка фиксирует параметры и рассчитанные коэффициенты для отчета.
095 lines = [
096     "Сводка по лабораторной работе 2",
097     f"Частота дискретизации: {SAMPLE_RATE} Гц",
098     f"Длительность: {DURATION_SEC} с",
099     "",
100     "Однородный фильтр:",
101     f"Размер окна: {HOMOGENEOUS_WINDOW}",
102     f"Коэффициенты h[n] = {np.array2string(h_homogeneous,
precision=6)}",
103     "",
104     "КИХ режекторный фильтр (P, прямоугольное окно):",
105     f"Полоса подавления: {NOTCH_LOW_HZ:.1f}..{NOTCH_HIGH_HZ:.1f} Гц",
106     f"Число коэффициентов: {NOTCH_TAPS}",
107     f"Центральный коэффициент h[M] = {h_notch[NOTCH_TAPS // 2]:.8f}",
108     f"Первые 10 коэффициентов = {np.array2string(h_notch[:10],
precision=8)}",
109     "",
110     "БИХ однополосный НЧ-фильтр:",
111     f"Частота среза: {ONE_POLE_CUTOFF_HZ:.1f} Гц",
112     f" $\alpha = \exp(-2\pi \cdot f_c / f_s) = \{alpha:.8f\}$ ",
113     "Разностное уравнение:  $y[n] = (1-\alpha) \cdot x[n] + \alpha \cdot y[n-1]$ ",
114 ]
115 (OUTPUT / "сводка_результатов.txt").write_text("\n".join(lines),
encoding="utf-8")
116
117 print("Лабораторная работа 2 выполнена.")
118 print(f"Результаты сохранены в: {OUTPUT}")
119
120
121 if __name__ == "__main__":
122     run()

```